

GAMING TIME ANALYZER

PLATFORM – VITYARTHI

**DOMAIN – PYTHON PROGRAMMING &
SOFTWARE DEVELOPMENT**

SUBMITTED BY – PRITHVIRAJ PATIDAR

REG. NO. – 26BA110787

INDEX

1.	INTRODUCTION-----	(2)
2.	PROBLEM STATEMENT-----	(3)
3.	FUNCTIONAL REQUIREMENTS-----	(4)
4.	NON-FUNCTIONAL-----	(5)
	REQUIREMENTS	
5.	SYSTEM ARCHITECTURE-----	(6)
6.	DESIGN DIAGRAMS-----	(8)
7.	DESIGN DECISION & RATIONALE-----	(13)
8.	IMPLEMENTATION DETAILS-----	(14)
9.	SCREENSHOTS / DETAILS-----	(15)
10.	TESTING APPROACH-----	(18)
11.	CHALLENGES FACED-----	(19)
12.	LEARNINGS & KEY TAKEAWAYS-----	(20)
13.	FUTURE ENHANCEMENTS-----	(21)
14.	REFERENCES-----	(22)

INTRODUCTION

Video gaming has evolved into one of the most engaging forms of digital entertainment. However, without active self-monitoring, extended screen sessions often lead to digital fatigue, disrupted routines, and unintended sedentary lifestyles.

The **Gaming Time Analyzer** is a modular Python-based tracking application developed to bridge the gap between recreational gaming and digital wellness. The system enables users to record individual gaming sessions, monitor total and average play durations, review usage trends across specific titles, and receive automated limit warnings when safe thresholds are breached. Built around core software engineering principles such as modular separation of concerns and persistent data management, the project serves as a lightweight, privacy-focused alternative to bulky background telemetry apps.

PROBLEM STATEMENT

Modern gamers deal with a problem when it comes to keeping track of their gaming and managing their habits:

1. **No place to track:** Gamers who use different platforms like Steam, Epic or standalone emulators don't have one place where they can see how much time they spend playing each day or each week.
2. **Not paying attention to time:** Players often forget how long they have been playing when they are really, into a game. This leads to habits that're hard to break.
3. **Much extra stuff:** Many tracking tools take up a lot of resources. They need to run in the background track activity or require a special account to use.

Goal: Create and build a program that works on a console uses Python. Does not need an internet connection. The program should record when sessions start and end keep game records safe show numbers and facts and warn when limits are reached.

Functional Requirements

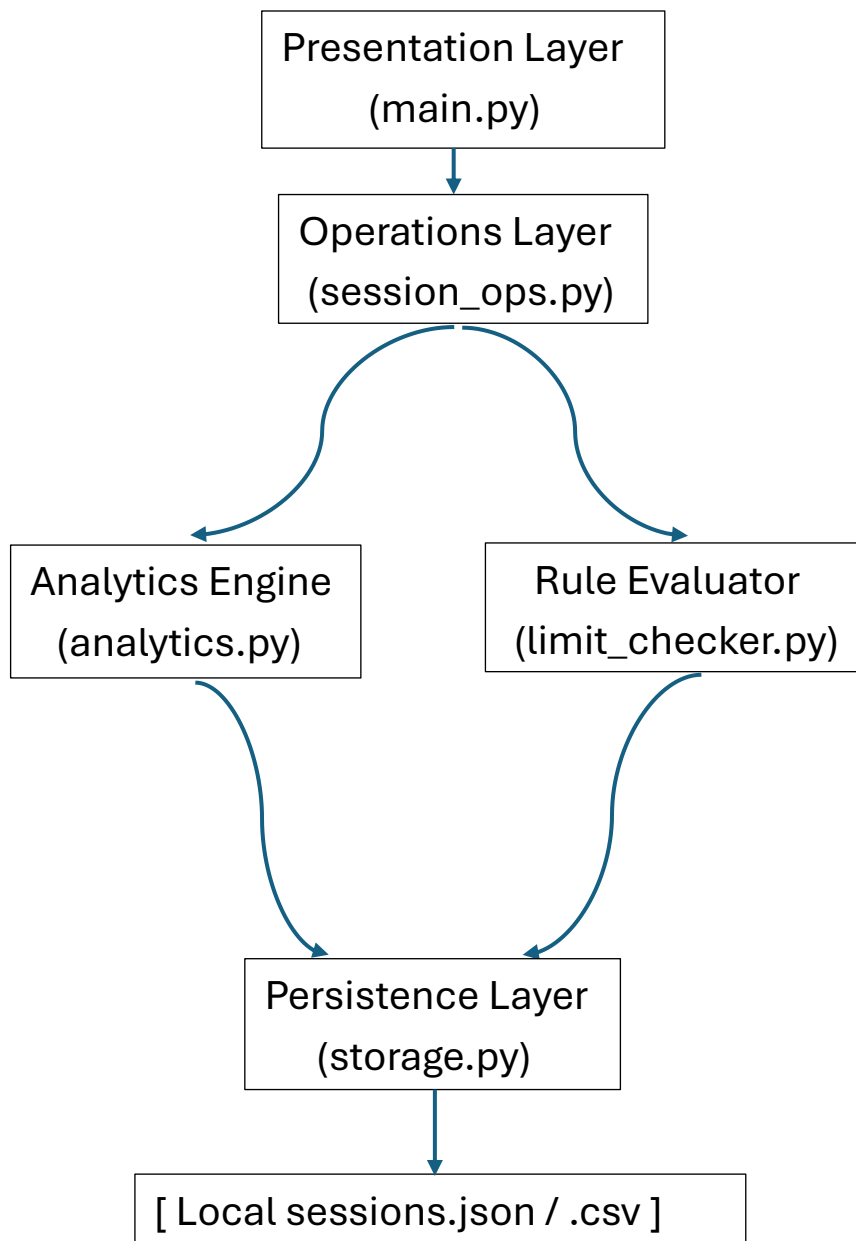
- **FR-1: Session Logging:** I need Session Logging so that users can record either a past gaming session. Users should provide the game title session start time and duration. The system can also automatically compute timestamps for start. Stop if users choose that option.
- **FR-2: Persistent Storage Management:** I need Persistent Storage Management so that Session records are stored locally as JSON or CSV. The system should automatically reload those records every time the application restarts ensuring no data is lost.
- **FR-3: Analytics & Metric Generation:** I want Analytics & Metric Generation so that the application can calculate statistics. These statistics should include playtime average session length, a breakdown of usage by title and the identification of peak gaming periods.
- **FR-4: Threshold & Limit Monitoring:** I need Threshold & Limit Monitoring so that users can set session-level playtime thresholds. Whenever the total time goes beyond these limits the system should issue a warning.
- **FR-5: Interactive Menu Interface:** I want Interactive Menu Interface so that the console interface includes user input loops, clear error handling and displays data in tables, for easy navigation.

Non-Functional Requirements

- **NFR-1: Modularity & Maintainability:** The codebase must follow the Single Responsibility Principle (SRP). This means that UI, operations, analytics, persistence and rules should each live in their separate modules.
- **NFR-2: Efficiency & Low Footprint:** The application should run in terminal environments. Query times should be almost instant under 200 milliseconds. CPU and RAM usage must stay very low.
- **NFR-3: Data Integrity & Fault Tolerance:** The persistence module must be able to manage corrupted, missing or malformed records. It must do so without crashing the program.
- **NFR-4: Portability:** The code must run on any Python 3.9+ setup, on Windows, macOS or Linux. It should not need third-party GUIs or external database libraries.

SYSTEM ARCHITECTURE

The application uses a 5-tier Modular Layered Architecture. This design keeps user interactions from the main analytical tasks and file storage. Each layer has its job making the system easier to manage and update. The separation helps improve performance and makes debugging simpler. It also allows for scalability as the system grows.

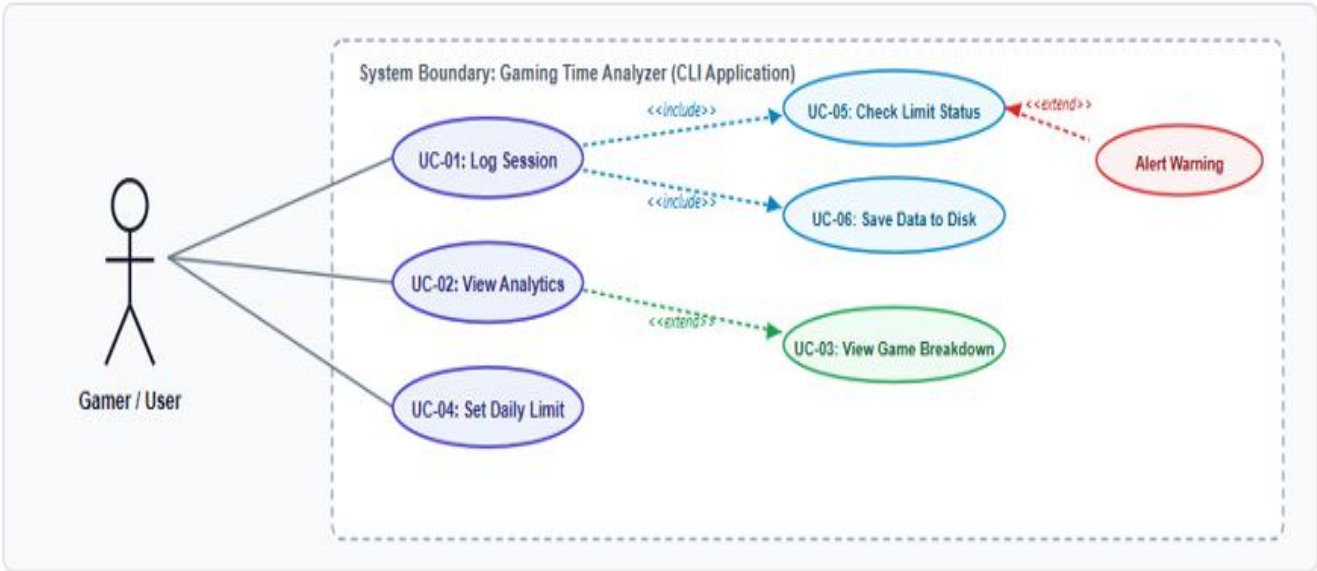


- **main.py:** This file takes care of the interface shows menus reads user input and directs everything to the right place.
- **session_ops.py:** This file starts sessions runs the timer and checks the records for correctness.
- **analytics.py:** This file does all the math, computing totals, averages and breaking down each game.
- **limit_checker.py:** This file compares each session to the safety limits that are set out beforehand.
- **storage.py:** This file handles reading and writing files making sure records are saved and loaded safely.

Design Diagrams

Use Case Diagram & Behavioral Model

The Use Case Diagram defines the behavioral interactions between the primary actor (**User / Gamer**) and the modular backend services of the **Gaming Time Analyzer**. It illustrates both the core functional workflows and automated system dependency relationships (`<<include>>` and `<<extend>>`).

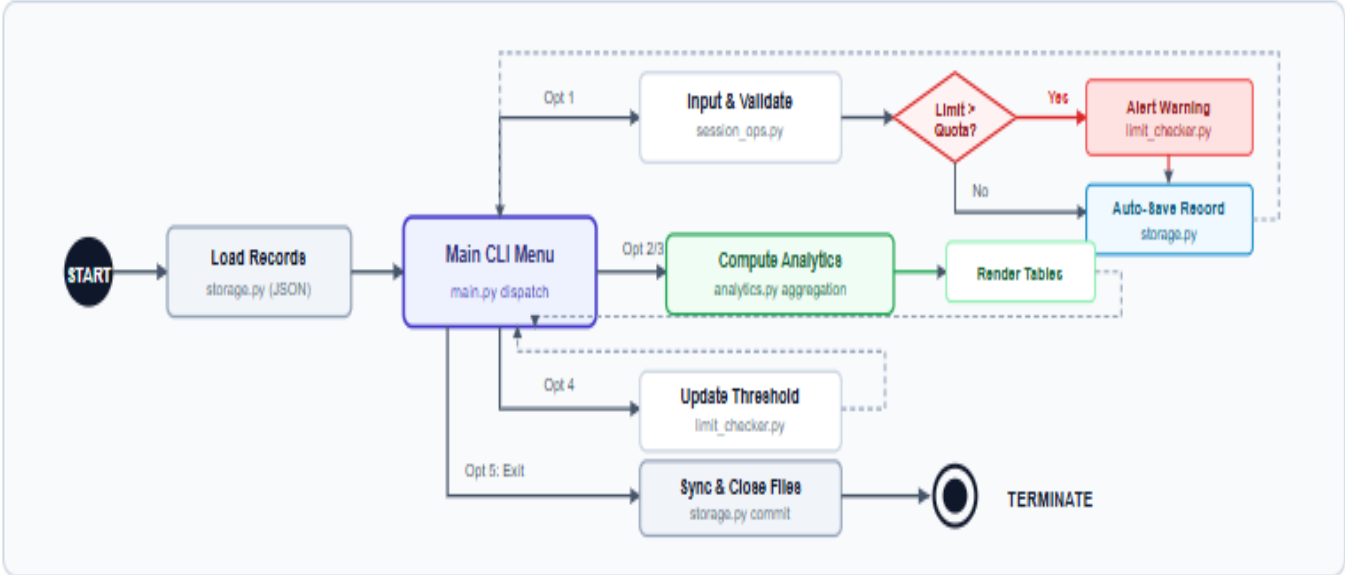


Use Case Specification Matrix (UC-01 to UC-06)

ID	Use Case Name	Target Module	Execution Behavior & Preconditions
UC-01	Log Gaming Session	<code>session_ops.py</code>	Captures title, date, duration. Automatically invokes storage and quota checking.
UC-02	View Overall Analytics	<code>analytics.py</code>	Calculates aggregate metrics (total playtime, average duration, session counts).
UC-03	View Game Breakdown	<code>analytics.py</code>	Itemizes cumulative duration and frequencies grouped by individual game titles.
UC-04	Set Daily Play Limit	<code>limit_checker.py</code>	Configures user-defined thresholds (in minutes) for maximum allowable gaming duration.
UC-05	Check Limit Status	<code>limit_checker.py</code>	Compares active daily playtime against set quota; triggers overtime warning if breached.
UC-06	Save Data to Disk	<code>storage.py</code>	Serializes session dictionaries to persistent <code>sessions.json</code> file storage.

Workflow Diagram & Operational Process Model

The Workflow Diagram captures the end-to-end procedural execution of the **Gaming Time Analyzer**. It illustrates application bootstrap, file-backed state restoration, interactive menu routing across functional modules, decision bifurcations, and persistent commit routines prior to graceful shutdown.

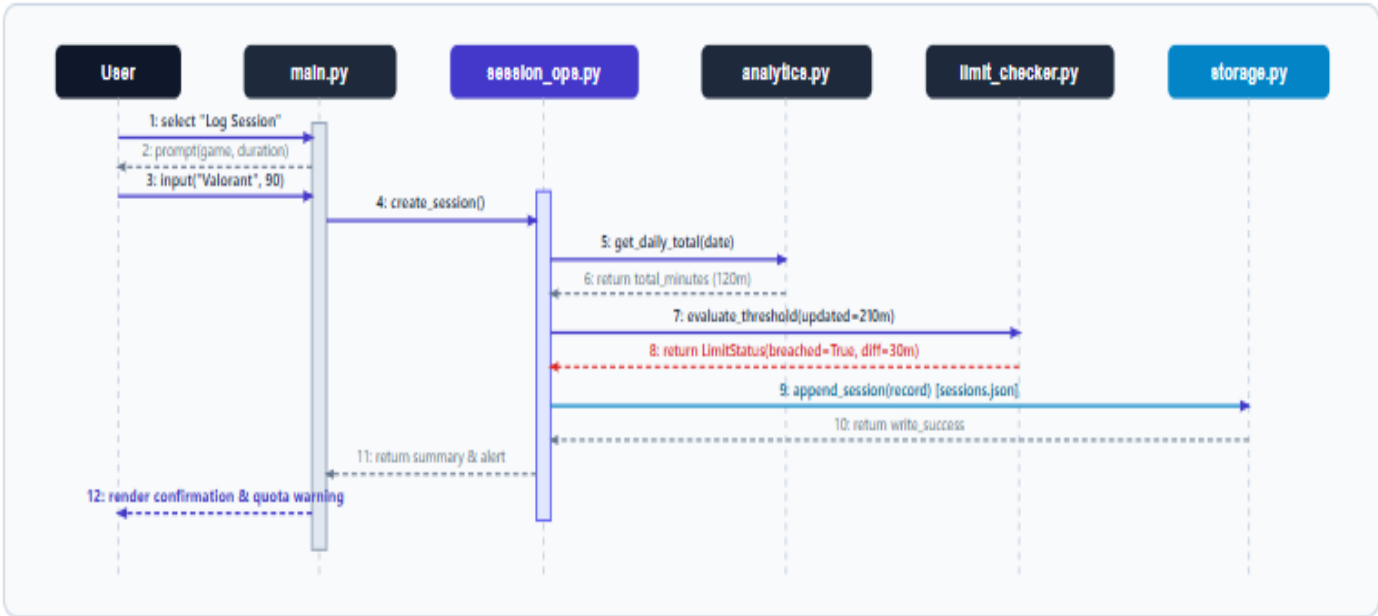


Workflow Execution Stages & State Transitions

Stage	Process Action	Participating File	Execution Mechanics & State Mutation
S-01	Startup & Deserialization	storage.py	Reads <code>sessions.json</code> . If missing, auto-initializes empty record schema safely.
S-02	Menu Loop & Routing	main.py	Renders options, sanitizes keystroke input, prevents non-integer selection crashes.
S-03	Session Processing	session_ops.py	Prompts game title/time, checks datetime bounds, validates non-negative duration.
S-04	Threshold Enforcement	limit_checker.py	Computes active day sum. If breached, raises instant alert before continuing.
S-05	Analytics Computation	analytics.py	Aggregates session sums, mean durations, and per-title groupings on demand.
S-06	Safe Exit & Commit	storage.py	Flushes pending memory buffers to disk and exits with code 0 cleanly.

Sequence Diagram & Dynamic Interaction Model

The Sequence Diagram documents the chronological message sequence and object lifelines during a gaming session logging operation. It highlights synchronous control transfers between the interactive CLI driver, validation operations, real-time analytics aggregation, threshold evaluation, and atomic file persistence.

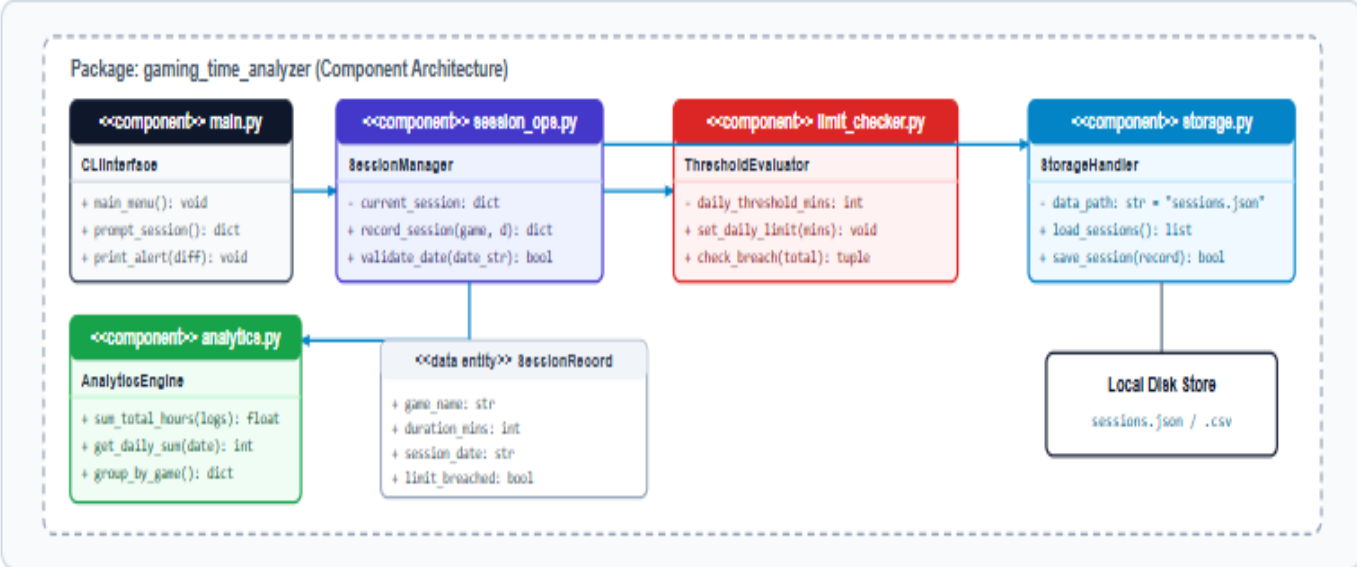


Chronological Execution Steps & Data Transfers

Step	Calling Component	Target Component	Message / Operation & Transferred Payload
01-03	User $\rightarrow$ <code>main.py</code>	<code>session_ops.py</code>	User selects session logging, inputs game title and duration; values sanitized.
04-06	<code>session_ops.py</code>	<code>analytics.py</code>	Queries prior day logs to compute updated daily playtime before saving.
07-08	<code>session_ops.py</code>	<code>limit_checker.py</code>	Compares cumulative time against threshold; generates warning flag if breached.
09-10	<code>session_ops.py</code>	<code>storage.py</code>	Commits formatted session dictionary to disk (<code>sessions.json</code>) atomically.
11-12	<code>session_ops.py</code> $\rightarrow$ <code>main.py</code>	User (Console)	Outputs formatted summary confirmation with high-visibility quota warning.

Class & Component Diagram Analysis

The Class and Component Diagram details the structural blueprint of the **Gaming Time Analyzer**. It establishes clear object boundaries, encapsulation levels, member attributes, methods, and component relationships across the presentation, operational orchestration, analytical calculation, quota evaluation, and persistence layers.

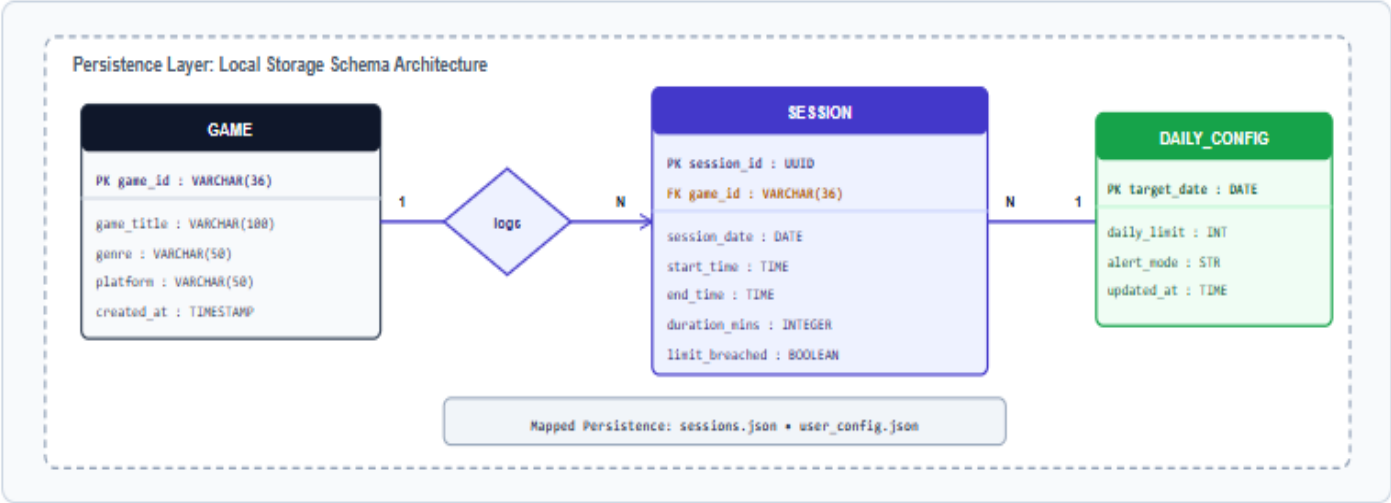


Class/Component Responsibility & Interaction Matrix

ID	Component & Class Name	Core Encapsulated Responsibility	Direct Technical Collaborations
C-01	CLIInterface (main.py)	Captures console input, validates menu routes, prints overtime alerts.	Delegates execution commands to SessionManager .
C-02	SessionManager (session_ops.py)	Orchestrates entry lifecycle, date validation, and pipeline coordination.	Invokes AnalyticsEngine , ThresholdEvaluator , and StorageHandler .
C-03	AnalyticsEngine (analytics.py)	Computes aggregate playtime, title rankings, and daily duration sums.	Provides statistical totals to SessionManager and CLI dashboard.
C-04	ThresholdEvaluator (limit_checker.py)	Maintains daily playtime targets, evaluates threshold breaches.	Returns boolean alert flags and overtime differentials to caller.
C-05	StorageHandler (storage.py)	Serializes and deserializes records safely to local disk storage.	Guarantees atomic file read/write operations for SessionRecord objects.

Entity-Relationship (ER) Diagram & Storage Schema

The Entity-Relationship (ER) model formalizes the persistent data schema utilized by the **Gaming Time Analyzer**. It illustrates entity definitions, primary and foreign key constraints, relationship cardinalities, and structural field mapping stored in local persistent files (`sessions.json` / `config.json`).



Data Dictionary & Schema Specification

Entity	Attribute Name	Data Type	Key / Constraint	Semantic Definition & Validation
GAME	game_id	VARCHAR(36)	PK Unique	Normalized identifier or canonical game slug.
GAME	game_title	VARCHAR(100)	NOT NULL	Display title of the video game (e.g., "Valorant").
SESSION	session_id	UUID / STR	PK Unique	Unique auto-generated record timestamp hash.
SESSION	game_id	VARCHAR(36)	FK Reference	Foreign key referencing parent record in GAME entity.
SESSION	duration_mins	INTEGER	CHECK (> 0)	Total active play duration in minutes.
DAILY_CONFIG	target_date	ISO DATE	PK Unique	Calendar date boundary for daily quota evaluation.
DAILY_CONFIG	daily_limit	INTEGER	DEFAULT 120	Daily threshold quota in minutes before alert trigger.

Design Decision & Rationale

1. Five-File Architecture versus Monolithic Script:

- Rationale: Putting everything in one script creates high coupling and makes automated testing hard. Separating analytics, storage and UI ensures any module can be changed (for example replacing JSON with an SQLite database) without changing business logic.

2. Text-Based File Persistence (JSON/CSV):

- Rationale: Lightweight JSON was chosen of heavy SQL servers because it does not need installation is easy for people to read and works very well with Python's native dictionary structures.

3. Console Interface versus Full GUI:

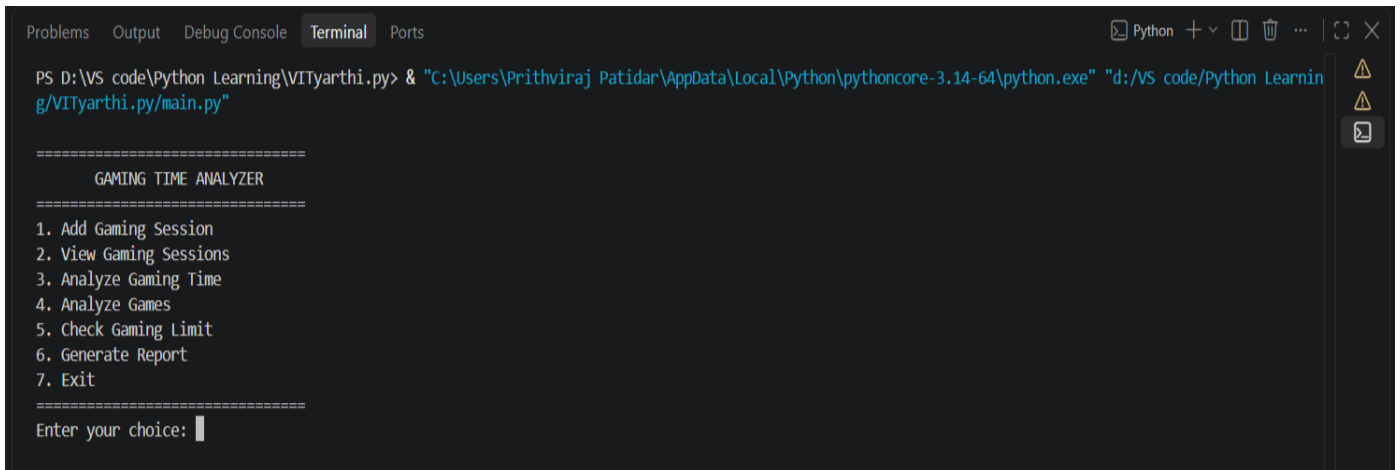
- Rationale: A console interface avoids third-party framework complications (like PyQt or Tkinter window event loops) making sure the utility stays fast, clear and free, from distractions.

Implementation Details

- **Language & Tools:** Python 3.x, Standard Libraries such as json, datetime, os, sys.
- **Key Implementation Highlights:**
 - Timestamp Parsing: Using datetime.strptime() and datetime.isoformat() to calculate elapsed minutes with accuracy avoiding any errors caused by timezone drift.
 - Defensive Input Handling: Every user prompt is protected by try-except blocks and regex or validators which stop errors that arise when a string is converted to an integer (ValueError).
 - Atomic File Writes: The storage module writes data snapshots that have been verified so that the file does not become corrupted if the terminal is closed unexpectedly.

Screenshots / Results

- Main Menu



```
PS D:\VS code\Python Learning\VITYarthi.py> & "C:\Users\Prithviraj Patidar\AppData\Local\Python\pythoncore-3.14-64\python.exe" "d:\VS code\Python Learning\VITYarthi.py/main.py"

=====
          GAMING TIME ANALYZER
=====
1. Add Gaming Session
2. View Gaming Sessions
3. Analyze Gaming Time
4. Analyze Games
5. Check Gaming Limit
6. Generate Report
7. Exit
=====
Enter your choice: |
```

- Adding 1st game



```
=====
          GAMING TIME ANALYZER
=====
1. Add Gaming Session
2. View Gaming Sessions
3. Analyze Gaming Time
4. Analyze Games
5. Check Gaming Limit
6. Generate Report
7. Exit
=====
Enter your choice: 1

--- Add Gaming Session ---
Enter date (DD-MM-YYYY): 25-09-2026
Enter game name: Valorant
Enter gaming time in hours: 3
Gaming session added successfully!
```

- Adding 2nd game



```
Problems Output Debug Console Terminal Ports
=====
          GAMING TIME ANALYZER
=====
1. Add Gaming Session
2. View Gaming Sessions
3. Analyze Gaming Time
4. Analyze Games
5. Check Gaming Limit
6. Generate Report
7. Exit
=====
Enter your choice: 1

--- Add Gaming Session ---
Enter date (DD-MM-YYYY): 25-09-2026
Enter game name: FreeFire
Enter gaming time in hours: 2
Gaming session added successfully!
```

- Analyse gaming time



```
Problems Output Debug Console Terminal Ports
2 . 25-09-2026 - FreeFire - 2.0 hours

=====
          GAMING TIME ANALYZER
=====

1. Add Gaming Session
2. View Gaming Sessions
3. Analyze Gaming Time
4. Analyze Games
5. Check Gaming Limit
6. Generate Report
7. Exit
=====

Enter your choice: 3

--- Gaming Time Analysis ---
Total gaming time: 5.0 hours
Average session time: 2.5 hours
```

- Check gaming limit

```
=====
GAMING TIME ANALYZER
=====
1. Add Gaming Session
2. View Gaming Sessions
3. Analyze Gaming Time
4. Analyze Games
5. Check Gaming Limit
6. Generate Report
7. Exit
=====
Enter your choice: 5

--- Gaming Limit Checker ---
Enter your gaming limit per session (hours): 2.5
25-09-2026 - Valorant : Limit exceeded!
25-09-2026 - FreeFire : Within limit.
```

- Generating report

```
=====
GAMING TIME ANALYZER
=====
1. Add Gaming Session
2. View Gaming Sessions
3. Analyze Gaming Time
4. Analyze Games
5. Check Gaming Limit
6. Generate Report
7. Exit
=====
Enter your choice: 6

===== GAMING REPORT =====
Number of sessions: 2
Total gaming time: 5.0 hours
Average session time: 2.5 hours

Sessions:
25-09-2026 - Valorant - 3.0 hours
25-09-2026 - FreeFire - 2.0 hours
```

Testing Approach

The application was tested manually by running scenarios across all five modules. The application was tested to verify data accuracy. The application was tested to verify input safety. The application was tested to verify alert triggering. The application was tested in each of the five modules to confirm data accuracy, input safety and alert triggering.

Test case	What Was tested	Input Given	Expected Output
Valid Session Entry	Normal session logging	Game: "Valorant" Duration: 3	Session saved, success message displayed
Negative Input	Preventing negative time	Duration: -2	Error: "Duration must be greater than 0", prompts again
Invalid Text Input	Preventing letter inputs for time	Duration: "abc"	Error: "Invalid number format", program does not crash
Daily Limit Alert	Overtime detection	Set limit: 2 hours Total played: 3 hours	Warning: "[Alert] Daily limit exceeded!"
Calculations Check	Math accuracy	Sessions: 1 hours and 2 hours	Total: 3 hours Average: 1.5 hours

Challenges Faced

1. Accurate Date-Time Normalization: Calculating daily totals when a single gaming session started at 11:30 PM and ended at 1:30 AM the next day.

- Accurate Date-Time Normalization: I handled cross-midnight splits by checking calendar date boundaries during session computation.

2. Data Consistency on Interruptions: Premature terminal exits during live sessions timers could produce dangling records.

- Data Consistency, on Interruptions: I implemented exit-signal interceptors and fallback manual logging modes.

Learning & Key Takeaways

- I have hands-on mastery of modular programming and strict separation of concern in Python.
- I understand state persistence, JSON schema validation and defensive programming, against inputs.
- I have gained experience translating a real-world problem screen-time management, into a functional data-driven software utility.

Future Enhancements

- **Automatic Game Process Detection:** Using Python's psutil library to find and track game programs that are running. This way playtime is recorded in the background without needing any input from the user.
- **Visual Graph Rendering:** Adding support for matplotlib or seaborn to create bar charts and trend graphs. These visualizations can be saved as PDF or PNG files, from the app.
- **Cloud Sync:** Offering an option to sync session data using Google Drive or GitHub Gist. This helps users keep their playtime records across devices without losing progress.

References

- Python Software Foundation. Python 3 Documentation (Modules: datetime, json, os).
<https://docs.python.org/3/>
- Class notes and material.
- Vityarthi Platform and documentation. [Vityarthi - Online Course & Education Platform](#)